sb-ecom API Documentation

sb-ecom — Spring Boot E-Commerce REST API

A full-featured e-commerce backend built with Spring Boot 4, providing REST APIs for product catalog, shopping cart, order management, and user authentication.

Tech Stack

Layer	Technology
Language	Java 17
Framework	Spring Boot 4.0.1
ORM	Spring Data JPA / Hibernate
Database	MySQL (AWS RDS-ready)
Auth	JWT (JJWT 0.13.0) via HTTP-only cookie
Security	Spring Security 6
Mapping	ModelMapper 3.2.4
Docs	SpringDoc OpenAPI / Swagger UI
Build	Maven

Project Structure

```
src/main/java/com/ecommerce/project/
├── controller/          # REST controllers (routes)
│   ├── AuthController.java
│   ├── ProductController.java
│   ├── CategoryController.java
│   ├── CartController.java
│   ├── OrderController.java
│   └── AddressController.java
├── model/               # JPA entity classes
│   ├── User.java
│   ├── Product.java
│   ├── Category.java
│   ├── Cart.java / CartItem.java
│   ├── Order.java / OrderItem.java
│   ├── Address.java
│   ├── Payment.java
│   └── Role.java / AppRole.java (enum)
├── payload/             # DTOs (request / response bodies)
│   ├── ProductDTO.java / ProductResponse.java
│   ├── CategoryDTO.java / CategoryResponse.java
│   ├── CartDTO.java / CartItemDTO.java
│   ├── OrderDTO.java / OrderItemDTO.java / OrderRequestDTO.java
│   ├── AddressDTO.java
│   ├── PaymentDTO.java
│   └── APIResponse.java
├── repositories/        # Spring Data JPA repositories
├── service/             # Business logic (interface + impl)
├── security/            # JWT filter, entry point, user details
│   ├── jwt/
│   ├── services/
│   ├── request/         # LoginRequest, SignupRequest
│   └── response/        # UserInfoResponse, MessageResponse
├── config/              # AppConfig, AppConstants, SwaggerConfig
├── exceptions/          # Global exception handler + custom exceptions
└── util/                # AuthUtil (current user helper)
```

Getting Started

Prerequisites

- Java 17+

- Maven 3.8+
- MySQL 8 running locally (or AWS RDS)

Configuration

Edit `src/main/resources/application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/ecommerce
spring.datasource.username=root
spring.datasource.password=your_password

spring.app.jwtSecret=your_jwt_secret_min_32_chars
spring.app.jwtExpirationMs=3000000
spring.ecom.app.jwtCookieName=springBootEcom
```

Run

```
mvn spring-boot:run
```

The API starts at `http://localhost:8080`.

Swagger UI

```
http://localhost:8080/swagger-ui/index.html
```

Data Models

User

Field	Type	Notes
userId	Long	PK
userName	String	unique, max 20
email	String	unique, valid email
password	String	BCrypt hashed
roles	Set<Role>	ROLE_USER / ROLE_SELLER / ROLE_ADMIN

Product

Field	Type	Notes
productId	Long	PK
productName	String	min 3 chars
image	String	file path
description	String	min 6 chars
quantity	Integer	
price	Double	
discount	Double	
specialPrice	Double	price after discount
category	Category	FK

Category

Field	Type
categoryId	Long
categoryName	String (min 5)

Address

Field	Type	Notes
addressId	Long	PK
street	String	min 5
buildingName	String	min 5
city	String	min 4
state	String	min 2
country	String	min 3
pincode	String	min 5

Cart / CartItem

A user has one cart. The cart holds many cart items, each linking a product with a quantity.

Order / OrderItem / Payment

An order captures the cart snapshot at checkout time, linked to an address and a payment record.

API Reference

> Authentication uses JWT stored in an HTTP-only cookie named `springBootEcom`.
> Endpoints under `/api/admin/**` require `ROLE_ADMIN`. All others require a valid JWT unless marked **Public**.

Auth — `/api/auth`

Method	Path	Auth	Body	Response
POST	`/api/auth/signup`	Public	`SignupRequest`	`MessageResponse`
POST	`/api/auth/signin`	Public	`LoginRequest`	`UserInfoResponse` + sets JWT cookie
POST	`/api/auth/signout`	Required	—	`MessageResponse` + clears cookie
GET	`/api/auth/username`	Required	—	`String`
GET	`/api/auth/user`	Required	—	`UserInfoResponse`

SignupRequest

```
{
  "username": "john",
  "email": "john@example.com",
  "password": "secret123",
  "role": ["user"]
}
```

LoginRequest

```
{
  "username": "john",
  "password": "secret123"
}
```

Categories — `/api`

Method	Path	Auth	Body	Response
GET	`/api/public/categories`	Public	—	`CategoryResponse` (paginated)
POST	`/api/public/categories`	Required	`CategoryDTO`	`CategoryDTO`
PUT	`/api/public/categories/{categoryId}`	Required	`CategoryDTO`	`CategoryDTO`
DELETE	`/api/admin/categories/{categoryId}`	Admin	—	`CategoryDTO`

Query params (GET): `pageNumber`, `pageSize`, `sortBy`, `sortOrder`
Defaults: page 0, size 10, sorted by `categoryId` ascending

Products — `/api`

Method	Path	Auth	Body	Response
GET	`/api/public/products`	Public	—	`ProductResponse` (paginated)
GET	`/api/public/categories/{categoryId}/products`	Public	—	`ProductResponse` (paginated)
GET	`/api/public/products/keyword/{keyword}`	Public	—	`ProductResponse` (paginated)
POST	`/api/admin/categories/{categoryId}/product`	Admin	`ProductDTO`	`ProductDTO`
PUT	`/api/admin/products/{productId}`	Admin	`ProductDTO`	`ProductDTO`
DELETE	`/api/admin/products/{productId}`	Admin	—	`ProductDTO`
PUT	`/api/products/{productId}/image`	Required	`multipart/form-data` (field: `image`)	`ProductDTO`

Query params (GET list): `pageNumber`, `pageSize`, `sortBy`, `sortOrder`

ProductDTO

```
{
  "productName": "Wireless Mouse",
  "description": "Ergonomic wireless mouse",
  "quantity": 100,
  "price": 29.99,
  "discount": 10.0
}
```

Cart — `/api`

Method	Path	Auth	Body	Response
POST	`/api/carts/products/{productId}/quantity/{quantity}`	Required	—	`CartDTO`
GET	`/api/carts/users/cart`	Required	—	`CartDTO`
GET	`/api/carts`	Admin	—	`List<CartDTO>`
PUT	`/api/cart/products/{productId}/quantity/{operation}`	Required	—	`CartDTO`
DELETE	`/api/carts/{cartId}/product/{productId}`	Required	—	`String`

`{operation}` is either `"add"` or `"delete"` (increments / decrements by 1).

CartDTO response

```
{
  "cartId": 1,
  "totalPrice": 59.98,
  "products": [
    {
      "productId": 3,
      "productName": "Wireless Mouse",
      "price": 29.99,
      "quantity": 2
    }
  ]
}
```

Orders — `/api`

Method	Path	Auth	Body	Response
POST	`/api/order/users/payments/{paymentMethod}`	Required	`OrderRequestDTO`	`OrderDTO`

OrderRequestDTO

```
{
  "addressId": 1,
  "paymentMethod": "CARD",
  "pgName": "Stripe",
  "pgPaymentId": "pi_abc123",
  "pgStatus": "SUCCESS",
  "pgResponseMessage": "Payment processed"
}
```

Addresses — `/api`

Method	Path	Auth	Body	Response
POST	`/api/addresses`	Required	`AddressDTO`	`AddressDTO`
GET	`/api/addresses`	Admin	—	`List<AddressDTO>`
GET	`/api/addresses/{addressId}`	Required	—	`AddressDTO`
GET	`/api/users/addresses`	Required	—	`List<AddressDTO>`
PUT	`/api/addresses/{addressId}`	Required	`AddressDTO`	`AddressDTO`
DELETE	`/api/addresses/{addressId}`	Required	—	`String`

AddressDTO

```
{
  "street": "123 Main St",
  "buildingName": "Apt 4B",
  "city": "Springfield",
  "state": "IL",
  "country": "USA",
  "pincode": "62701"
}
```

Pagination Response Format

List endpoints return a paginated envelope:

```
{
  "content": [...],
  "pageNumber": 0,
  "pageSize": 10,
  "totalElements": 42,
  "totalPages": 5,
  "lastPage": false
}
```

Security Model

- JWT stored in HTTP-only cookie (`springBootEcom`), 50-minute expiry
- Stateless sessions (no server-side session storage)
- Role hierarchy: `ROLE_USER` → `ROLE_SELLER` → `ROLE_ADMIN`
- Passwords hashed with BCrypt
- Public paths: `/api/auth/**`, `/api/public/**`, `/swagger-ui/**`, `/v3/api-docs/**`, `/images/**`

Roles

Role	Capabilities
`ROLE_USER`	Browse products, manage own cart, place orders, manage own addresses
`ROLE_SELLER`	All of USER + add/update own products
`ROLE_ADMIN`	Full access including category/product deletion, view all carts/addresses

Image Uploads

Product images are stored locally under the `images/` directory (configured via `project.image` property). Upload via `PUT /api/products/{productId}/image` with `multipart/form-data`.

AWS Deployment Notes

The `application.properties` includes commented-out AWS RDS and Elastic Beanstalk configuration:

```
# AWS RDS
spring.datasource.url=jdbc:mysql://<rds-endpoint>:3306/mysqlecomdb
spring.datasource.username=ecomdb
spring.datasource.password=your_rds_password

# Elastic Beanstalk port
server.port=5000
```